University of Engineering and Technology, Lahore

MR_Lab_Report_5

Submitted by: 2022-MC_27

Introduction to Gazebo and RViz with ROS 2 using TurtleBot3

1. Objectives

By the end of this lab session, students will be able to:

- Launch and navigate the TurtleBot3 robot in a Gazebo simulated environment.
- Use RViz to visualize sensor data including LiDAR scans, odometry, TF frames, and maps.
- Operate the robot using keyboard teleoperation commands.
- Build a map of the simulated environment using the Cartographer SLAM package.
- Record and replay robot motion data using the ros2 bag tool.
- Write and run a ROS 2 publisher node that controls robot velocity via the /cmd_vel topic.
- Write and run a ROS 2 subscriber node that reads and prints odometry data from /odom.
- Understand the relationship between ROS 2 nodes, topics, and message types in a real simulation context.

2. Introduction

This lab session introduces students to two of the most commonly used tools in the ROS 2 ecosystem for robot simulation and visualization: Gazebo and RViz. These tools are used together to simulate, control, and analyze a mobile robot without requiring any physical hardware, making them essential for prototyping and testing robot software.

The robot platform used in this lab is the TurtleBot3 Burger, a compact, low-cost, differential-drive mobile robot developed by ROBOTIS. It is widely used in research and academic settings due to its compatibility with ROS 2 and its built-in support for sensors such as LiDAR and IMU. The TurtleBot3 is available in two variants — Burger and Waffle — and this lab primarily uses the Burger model.

Gazebo is an open-source 3D robotics simulator that provides realistic physics simulation, sensor modeling, and seamless integration with ROS 2 through the gazebo_ros_pkgs interface. It allows a robot model to exist in a virtual world complete with gravity, friction, and obstacles, giving developers a safe environment to test algorithms.

RViz (Robot Visualization) is a separate ROS 2 tool for real-time 3D visualization of robot state and sensor data. Unlike Gazebo, RViz does not simulate physics — instead, it displays data being published on ROS 2 topics, such as laser scan readings, coordinate frames, odometry paths, and generated maps. Together, Gazebo and RViz give a comprehensive view of what a robot is doing and perceiving.

This lab also introduces the Cartographer package, a Google-developed SLAM (Simultaneous Localization and Mapping) library that enables the robot to build a map of its environment while tracking its own position within that map. The lab concludes with two programming tasks that require writing ROS 2 nodes in Python.

3. Prerequisites and Environment Setup

3.1 Prerequisites

- Basic knowledge of Linux terminal commands.
- Familiarity with ROS 2 core concepts: nodes, topics, publishers, subscribers, and services.
- ROS 2 Humble Hawksbill installed and sourced on the system.
- TurtleBot3 and Gazebo ROS packages installed.

3.2 Package Installation

Before starting the lab, the required packages were installed using the following commands:

```
sudo apt update
sudo apt install ros-humble-turtlebot3* ros-humble-gazebo-ros-pkgs
```

3.3 Environment Configuration

The TurtleBot3 model environment variable was set to 'burger' so all launch files know which model to use. This line was also added to ~/.bashrc to persist across terminal sessions.

```
export TURTLEBOT3_MODEL=burger
source /opt/ros/humble/setup.bash
```

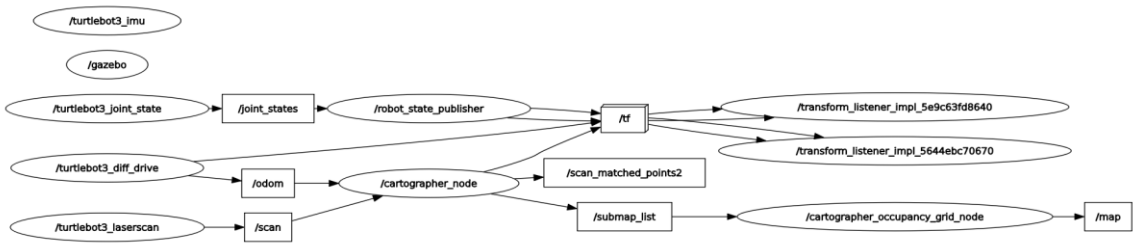
4. Procedure

4.1 Launching the Gazebo Simulation

The first step was to launch the TurtleBot3 simulation in Gazebo. This command starts the simulated world and spawns the TurtleBot3 robot model inside it. Gazebo opens with a pre-built world environment that includes walls and obstacles.

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

After the simulation loaded, the rqt_graph tool was used to observe all active ROS 2 nodes and the topics connecting them. This provides a clear picture of the communication structure within the simulation.



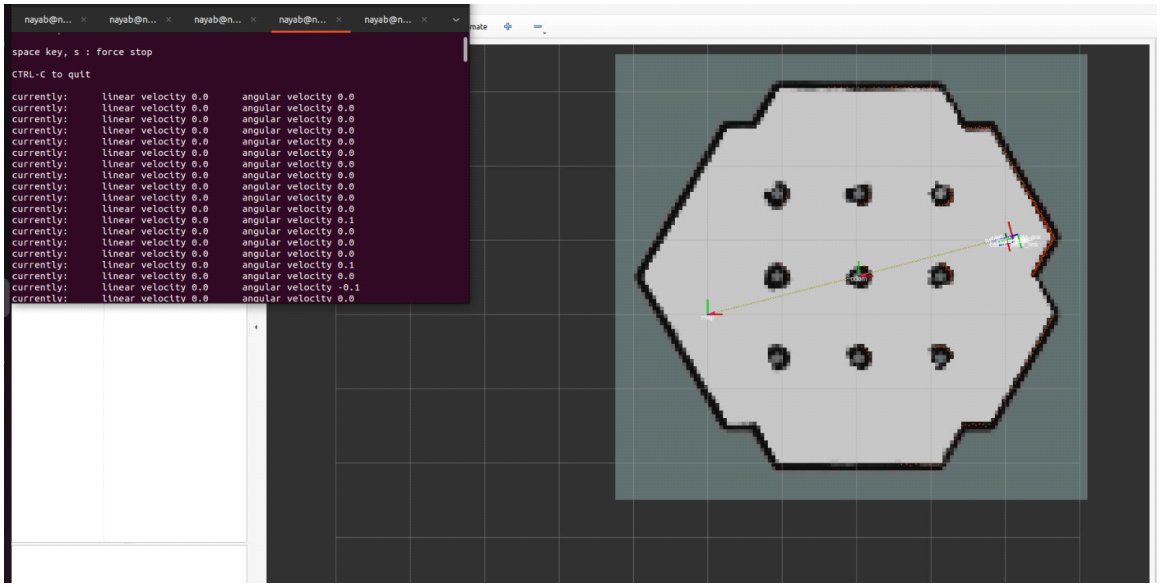
4.2 Launching RViz with Cartographer (SLAM)

In a new terminal, the Cartographer SLAM package was launched alongside RViz. This opens the RViz visualization window and begins mapping the environment in real time as the robot moves. The use_sim_time:=true flag ensures that all nodes use simulated time from Gazebo rather than the system clock, which is critical for synchronizing sensor data.

```
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=true
```

The RViz window displayed the following data streams live:

- Laser scan data from the LiDAR sensor (displayed as red dots around the robot).
- Odometry data from the /odom topic (tracks robot position and orientation).
- The map being constructed incrementally by the Cartographer algorithm.



4.3 Exploring RViz Visualization Plugins

The following visualization plugins were added and configured within RViz using the Add button in the Displays panel:

Plugin	Topic / Source	Purpose
LaserScan	`/scan`	Displays LiDAR point cloud around robot
TF	`All frames`	Shows coordinate frame tree and relationships
Odometry	`/odom`	Visualizes robot position and motion path
Path	`/plan`	Displays planned or traveled navigation path
Map	`/map`	Renders SLAM-generated occupancy grid map

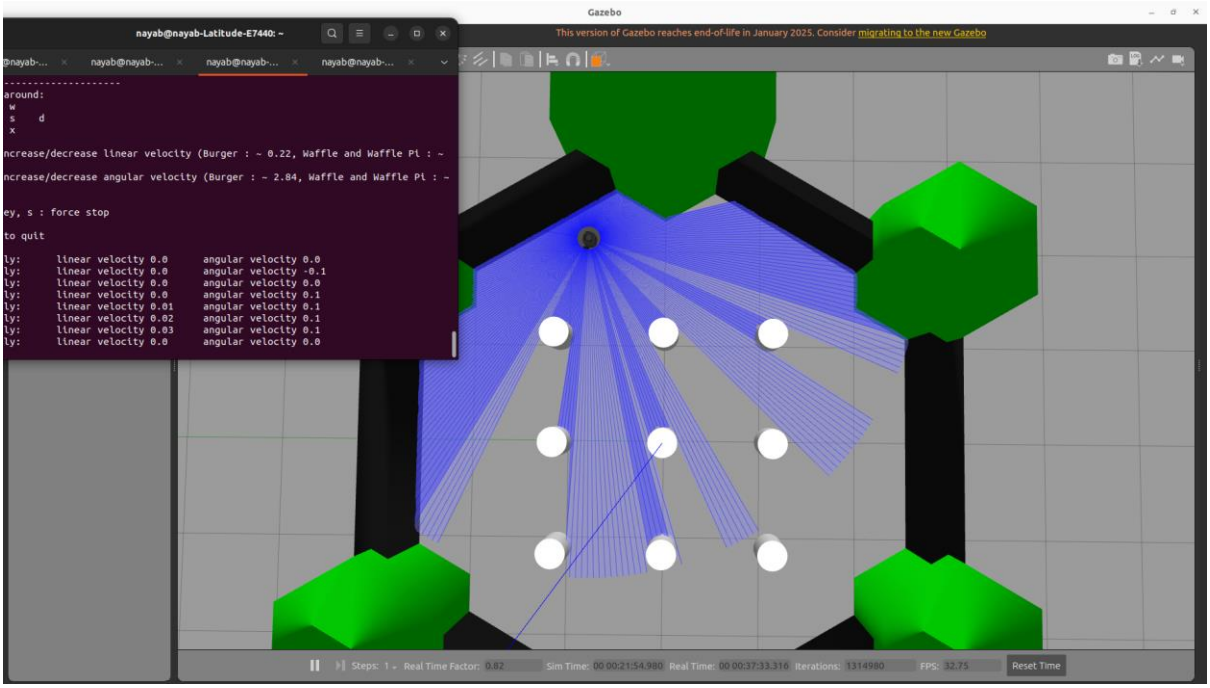
The Fixed Frame in RViz Global Options was set to map so that all data is aligned with the world coordinate frame. Color and size settings were adjusted to make the LiDAR scan and odometry arrows clearly visible.

Task 1: Navigate the Robot Using Teleoperation

In this task, the TurtleBot3 robot was manually controlled in the Gazebo simulation environment using the teleoperation keyboard node. The teleoperation node was launched using the command ros2 run turtlebot3_teleop teleop_keyboard, which allowed real-time control of the robot by publishing velocity commands to the /cmd_vel topic.

Using the keyboard controls (W, A, S, D), the robot was driven forward, backward, and rotated left and right within the simulated environment. The speed of the robot was also adjusted using the Q and Z keys. During navigation, the robot successfully moved from its initial spawn position while avoiding obstacles present in the Gazebo world.

The movement of the robot was continuously observed in both Gazebo and RViz. In RViz, the /odom topic displayed the robot's changing position and orientation, while the LaserScan data helped in detecting nearby obstacles. This task demonstrated basic manual navigation and familiarization with robot control using ROS 2 teleoperation.

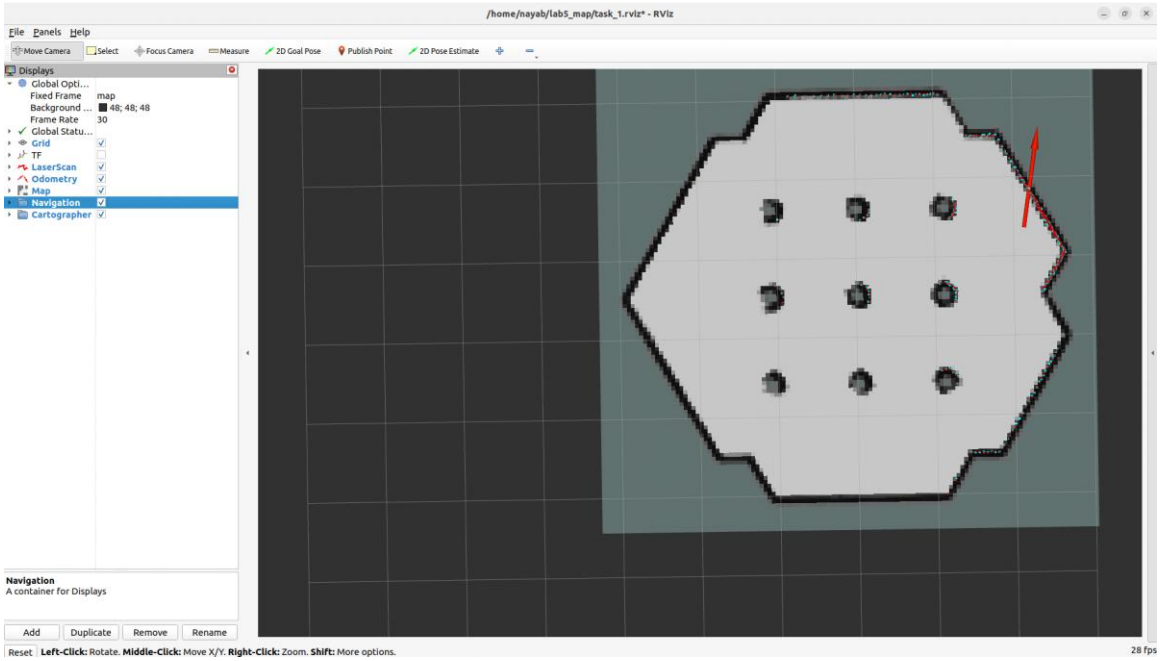


Task 2: Observe LiDAR Scan and Odometry Data in RViz

With the LaserScan and Odometry plugins active in RViz, the robot was driven around the environment and the sensor data was observed live.

LiDAR Scan observations: The LaserScan plugin rendered the laser data as a ring of colored points around the robot. When the robot approached a wall, the scan points compressed toward the robot model from that direction, accurately reflecting the distance to the obstacle. Open areas showed sparse or no scan returns, as the laser beams found no nearby surface.

Odometry observations: The Odometry plugin displayed the robot's estimated position and orientation as an arrow in the 3D world. As the robot moved, the arrow updated its direction and position. A trail of smaller arrows showed the history of positions visited — forming a path visualization. The pose data (x, y, z position and quaternion orientation) was simultaneously visible in the Odometry display properties.



Task 3: Explore the TF Plugin — Coordinate Frame Relationships

Theoretical Answer:

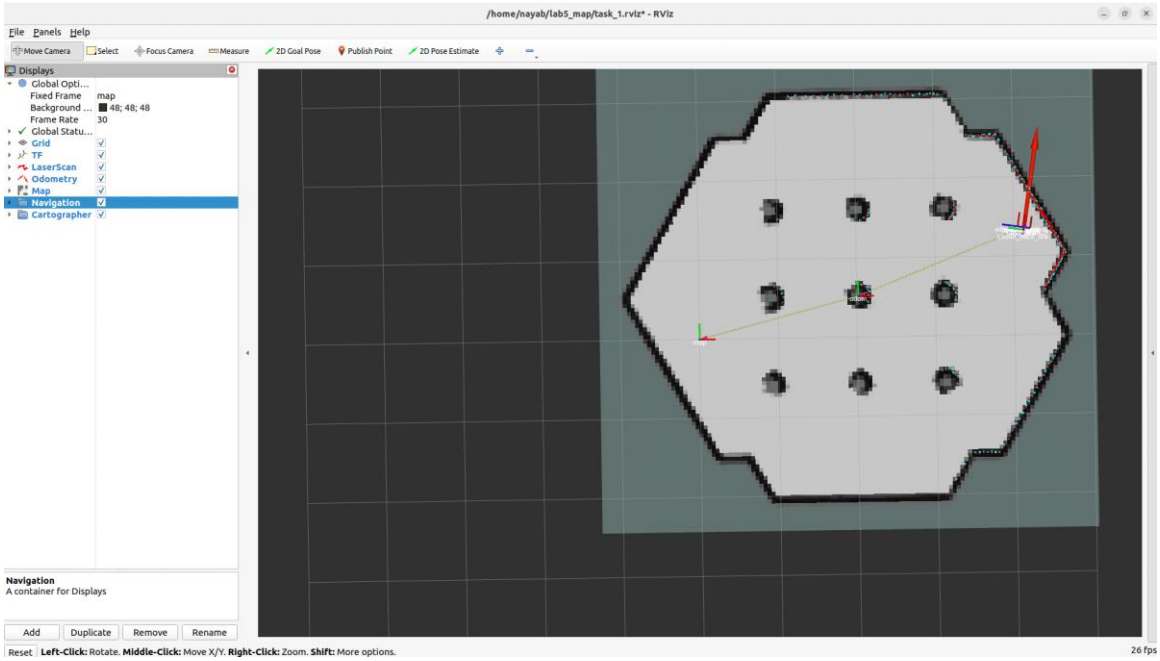
TF stands for Transform. In ROS 2, the TF system keeps track of all coordinate frames in the robot's world and the geometric transformations (translations and rotations) between them. Every physical part of the robot and its environment is assigned a named frame, and TF continuously broadcasts how each frame is positioned relative to every other frame.

For the TurtleBot3 in this simulation, the major TF frames observed were:

- map — The global world frame. The origin (0, 0, 0) of the entire map. Set as fixed frame in RViz.
- odom — The odometry frame. Tracks the robot's pose relative to where it started. It drifts over time due to wheel slip and sensor noise.
- base_footprint — A 2D projection of the robot onto the floor plane.
- base_link — The center reference frame of the robot body.
- base_scan — The frame attached to the LiDAR sensor mounted on top of the robot.
- imu_link — The frame for the onboard IMU sensor.

Frame relationships: The map frame is the parent of the odom frame. The odom frame is the parent of base_footprint. The base_footprint is the parent of base_link, which in turn is the parent of sensor frames like base_scan and imu_link. This hierarchy means that when the robot moves, updating the transform from odom to base_footprint automatically updates the positions of all child frames (including sensors) in the global map frame.

The TF plugin rendered each frame as a colored axis (red for X, green for Y, blue for Z) connected by lines showing the parent-child relationships. Watching the frames move in real time as the robot was driven demonstrated how the transform tree updates continuously.

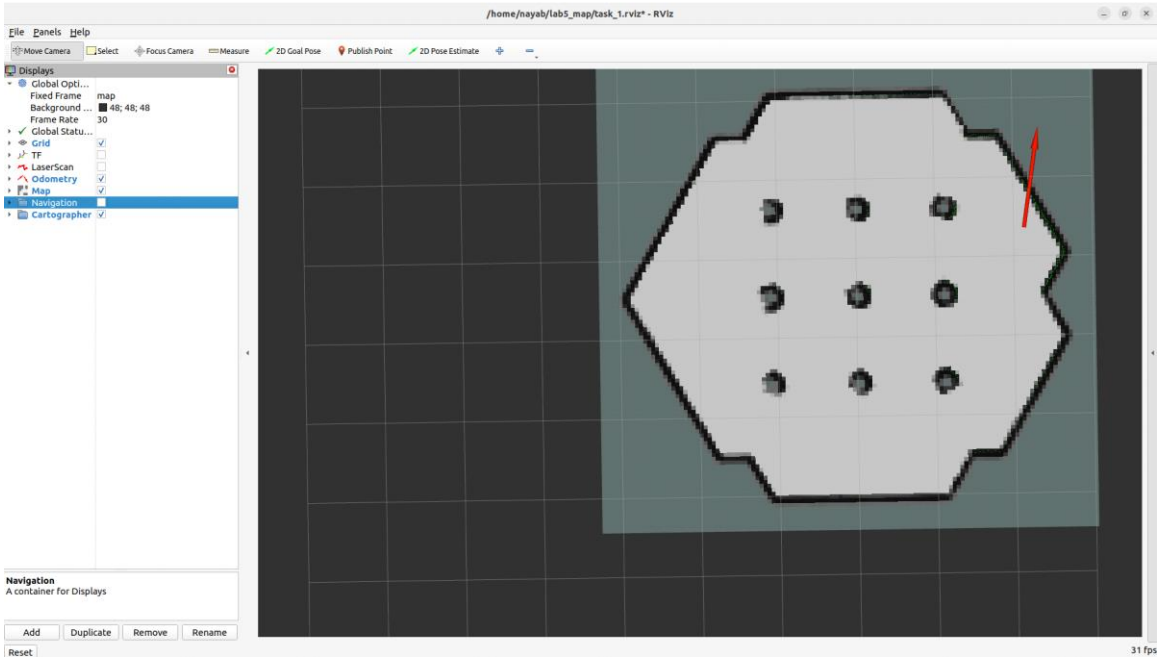


Task 4: Enable and Customize Odometry Display

The Odometry display plugin was enabled in RViz and customized to improve visibility of the robot's real-time motion. The following customizations were applied:

- Topic set to /odom to subscribe to the correct data source.
- Arrow shape selected to display pose as directional arrows.
- Keep set to a limited number (e.g., 50 poses) to show a meaningful history trail without cluttering the view.
- Color adjusted for clear contrast against the map background.

As the robot was driven using teleoperation, the odometry arrows updated in real time showing both current position and the accumulated path. Any change in direction was clearly reflected in the arrow orientations.



4.4Teleoperation

Keyboard teleoperation was used to manually drive the robot around the simulated environment. The teleop node listens for keyboard inputs and publishes corresponding velocity commands to the /cmd_vel topic.

```
ros2 run turtlebot3_teleop teleop_keyboard
```

The keyboard controls used were:

Key	Action
w	Move forward
s	Move backward / Stop forcefully
a	Turn left
d	Turn right
q	Increase speed
z	Decrease speed

5. Tasks — Detailed Execution and Observations

Task5:Record Robot Motion Using ros2 bag

The ros2 bag tool was used to record all active ROS 2 topics during the robot's motion. This creates a binary log file that can be replayed later for analysis or visualization.

Command used to start recording:

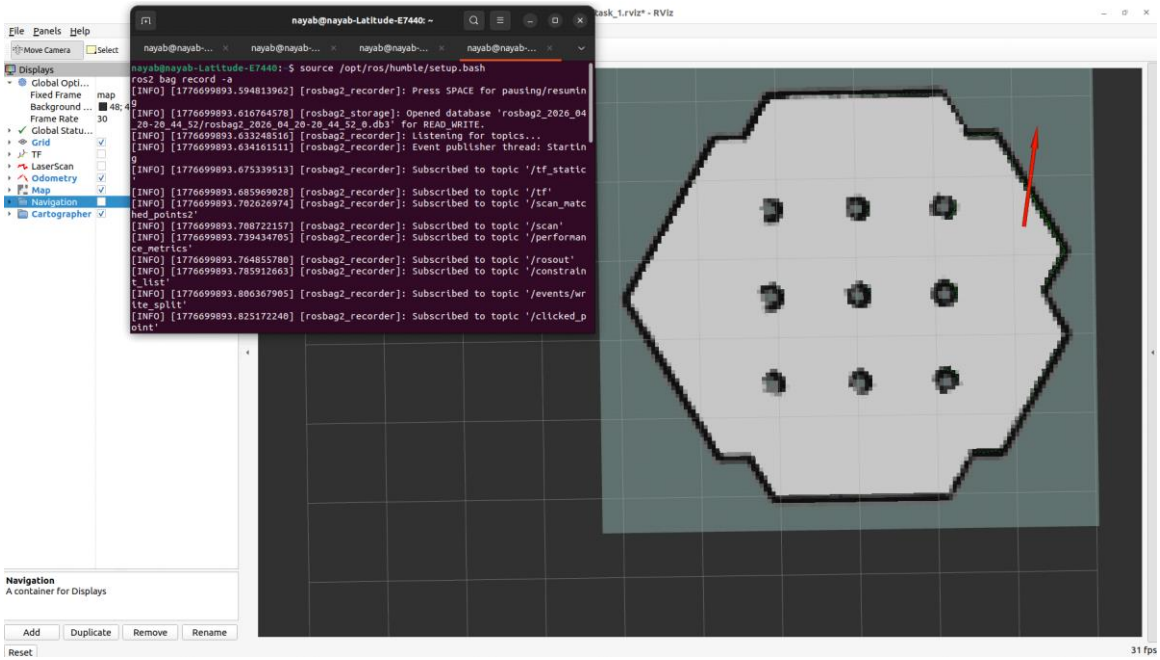
```
ros2 bag record -a
```

The recording was started before driving the robot, and stopped (Ctrl+C) after completing a short navigation run. The bag file was saved in the current working directory with an auto-generated timestamped folder name.

To replay and visualize the recorded data in RViz, the following command was used:

```
ros2 bag play <bag_folder_name>
```

During playback, RViz reproduced the robot's motion path, laser scans, and odometry exactly as they were recorded. This demonstrated that ros2 bag captures the complete state of all topic messages, making it a powerful tool for debugging and post-analysis.



Task 6: Save the SLAM-Generated Map Using Cartographer

While the robot was navigated around the environment with Cartographer running, the map was being built progressively. Once a satisfactory portion of the environment was mapped, the map was saved to disk using the nav2_map_server utility.

The following commands were used to create a maps directory and save the map:

```
cd
mkdir maps
ros2 run nav2_map_server map_saver_cli -f maps/my_map
```

This produced two files: my_map.pgm (the grayscale occupancy grid image) and my_map.yaml (the metadata file containing resolution, origin, and threshold values). The PGM image encodes the map as pixels — white for free space, black for obstacles, and gray for unknown areas.

Task 7:Navigate the Robot to a Designated Target Point

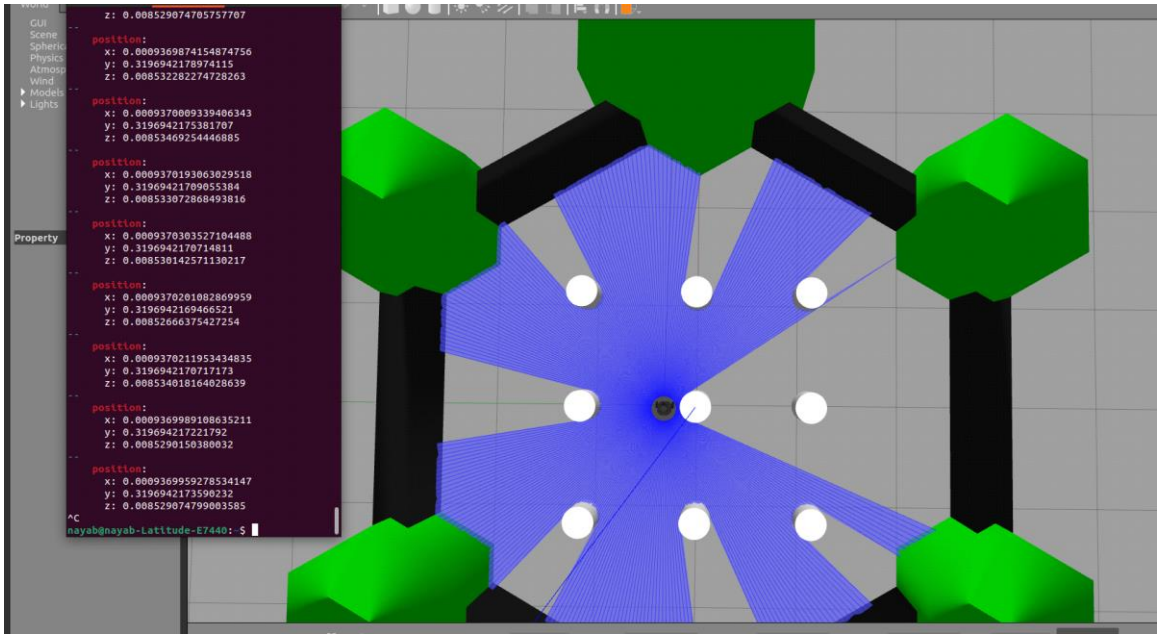
Using the keyboard teleoperation node, the robot was navigated from its starting position (0, 0, 0) to a designated target point within the Gazebo world. The target location was confirmed by observing the position values displayed in the RViz Odometry plugin and the robot's visual position in Gazebo.

During navigation, the robot's movement was clearly reflected in the /odom data stream in RViz. The robot's heading and position updated in real time, allowing precise maneuvering. Care was taken to avoid collision with the environment walls by monitoring the LiDAR scan cloud displayed in RViz.

Task 7: Teleop the TurtleBot3 Back to (0, 0, 0)

After navigating to various points in the map, the robot was manually driven back to its starting coordinates (0, 0, 0) using the keyboard teleoperation node. The target position was confirmed by monitoring the /odom topic data, which was visible through the Odometry plugin in RViz and later confirmed via the odom subscriber node created in Task 9.

This task demonstrated an important practical challenge: returning to the origin using only odometry is imprecise due to cumulative drift in wheel encoder data and IMU readings. In a real robot or a more advanced lab, this would be addressed using SLAM-based localization or a global positioning correction. In simulation, the drift is minimal, and the robot was able to return close to (0, 0, 0) using careful teleoperation.



Task 8: Create a /cmd_vel Publisher with Timer Callback

Code Explanation:

A ROS 2 Python publisher node was created that alternates between publishing a forward velocity command and a zero velocity (stop) command every 2 seconds. This simulates a simple start-stop motion pattern. Below is the complete code:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class CmdVelToggle(Node):

    def __init__(self):
        super().__init__('cmd_vel_toggle')

        # Publisher to /cmd_vel
        self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)

        # Timer fires every 2 seconds
        self.timer = self.create_timer(2.0, self.timer_callback)

        # State flag: True = move, False = stop
        self.move = True

        self.get_logger().info('CmdVel Toggle Node Started')

    def timer_callback(self):
        msg = Twist()

        if self.move:
            msg.linear.x = 0.2   # Move forward at 0.2 m/s
            msg.angular.z = 0.0
            self.get_logger().info('Moving Forward')
        else:
            msg.linear.x = 0.0   # Stop
            msg.angular.z = 0.0
            self.get_logger().info('Stopping')
```

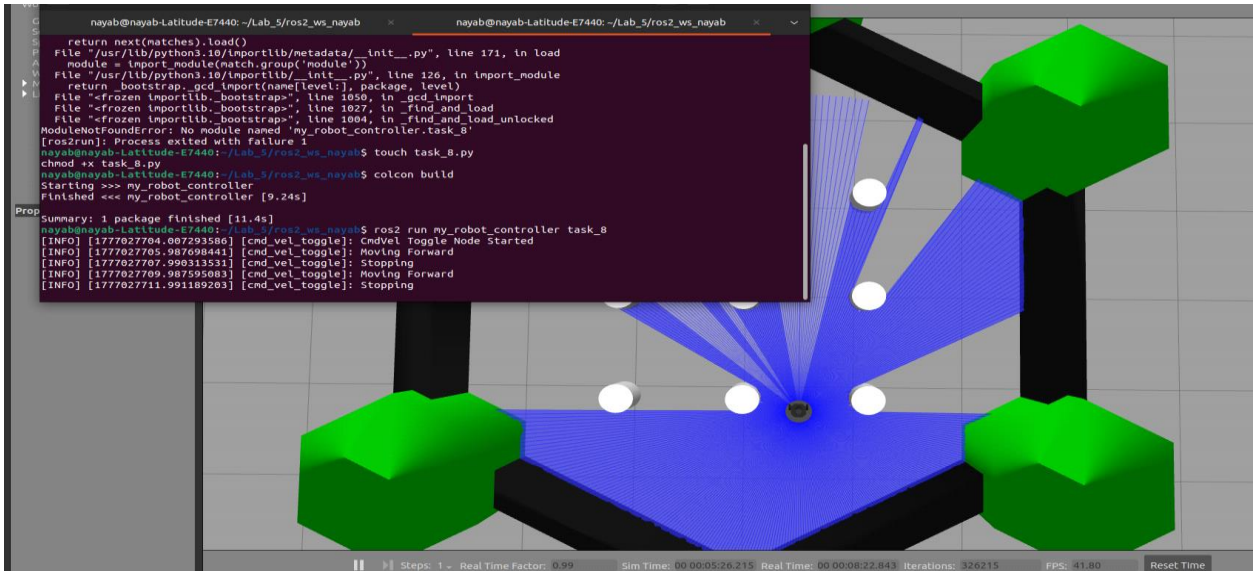
```
            self.publisher_.publish(msg)  # Send the message
            self.move = not self.move      # Toggle state

    def main(args=None):
        rclpy.init(args=args)
        node = CmdVelToggle()
        rclpy.spin(node)
        node.destroy_node()
        rclpy.shutdown()

    if __name__ == '__main__':
        main()
```

Code Breakdown:

- Message Type: geometry_msgs/msg/Twist. This message contains two 3D vectors: linear (x, y, z) for translational velocity and angular (x, y, z) for rotational velocity. For a ground robot, only linear.x (forward speed) and angular.z (turning rate) are used.
- Publisher: Created using create_publisher() targeting the /cmd_vel topic with a queue size of 10.
- Timer: create_timer(2.0, callback) fires the callback function every 2 seconds autonomously.
- State Toggle: The self.move boolean is flipped after each publish call, alternating between forward motion and stop.
- linear.x = 0.2: Sets forward velocity to 0.2 meters per second.
- linear.x = 0.0 and angular.z = 0.0: Fully stops the robot.

Running Output:

```
[INFO] [cmd_vel_toggle]: CmdVel Toggle Node Started
[INFO] [cmd_vel_toggle]: Moving Forward
[INFO] [cmd_vel_toggle]: Stopping
[INFO] [cmd_vel_toggle]: Moving Forward
[INFO] [cmd_vel_toggle]: Stopping
[INFO] [cmd_vel_toggle]: Moving Forward
...
```



Task 9: Create a /odom Subscriber and Print Received Messages

Message Type Identification:

Before writing the subscriber, the message type published to the /odom topic was identified using the following command:

```
ros2 topic info /odom
```

Output confirmed the message type as nav_msgs/msg/Odometry. This message contains two main sections:

- pose — The position (x, y, z) and orientation (quaternion: x, y, z, w) of the robot in the world frame.
- twist — The current linear and angular velocities of the robot.

Code Explanation:

A ROS 2 Python subscriber node was created that subscribes to the /odom topic and prints both the position and orientation of the robot whenever a new message arrives. Below is the complete code:

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry

class OdomSubscriber(Node):

    def __init__(self):
        super().__init__('odom_subscriber')

        self.subscription = self.create_subscription(
            Odometry,
            '/odom',
            self.callback,
            10
        )

        self.get_logger().info('Odom Subscriber Node Started')

    def callback(self, msg):
        # Extract position
        x = msg.pose.pose.position.x
        y = msg.pose.pose.position.y
        z = msg.pose.pose.position.z

        # Extract orientation (quaternion)
        ox = msg.pose.pose.orientation.x
        oy = msg.pose.pose.orientation.y
        oz = msg.pose.pose.orientation.z
        ow = msg.pose.pose.orientation.w

        self.get_logger().info(
            f'Position -> x: {x:.2f}, y: {y:.2f}, z: {z:.2f}'
        )
        self.get_logger().info(
            f'Orientation -> x: {ox:.2f}, y: {oy:.2f}, z: {oz:.2f}, w: {ow:.2f}'
        )

def main():
    rclpy.init()
```

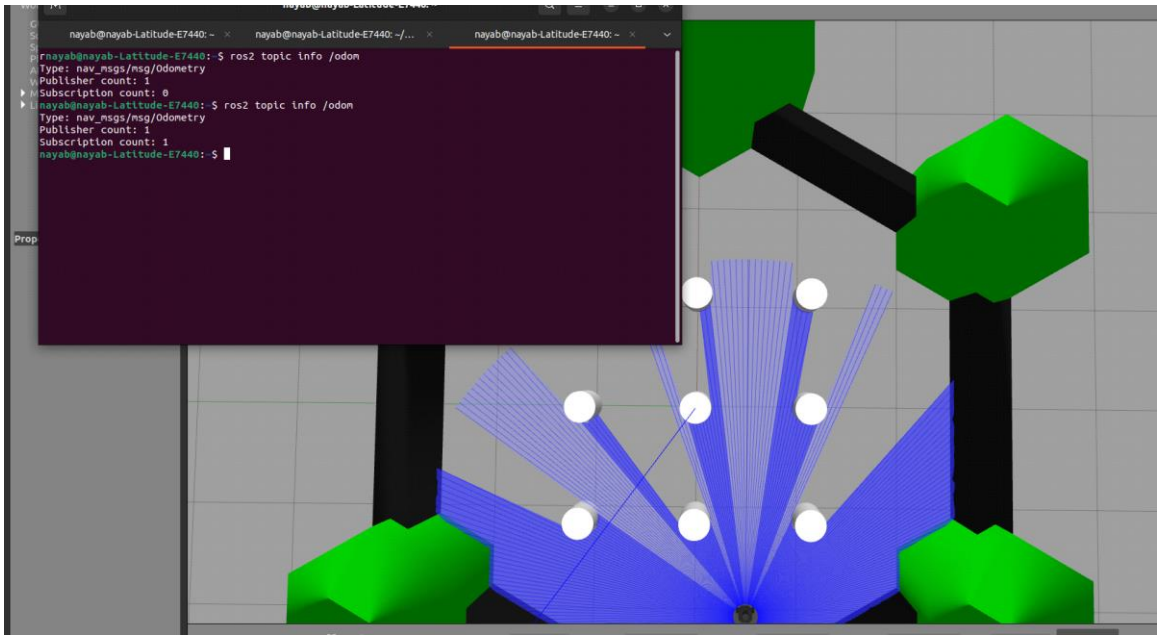
```
    node = OdomSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

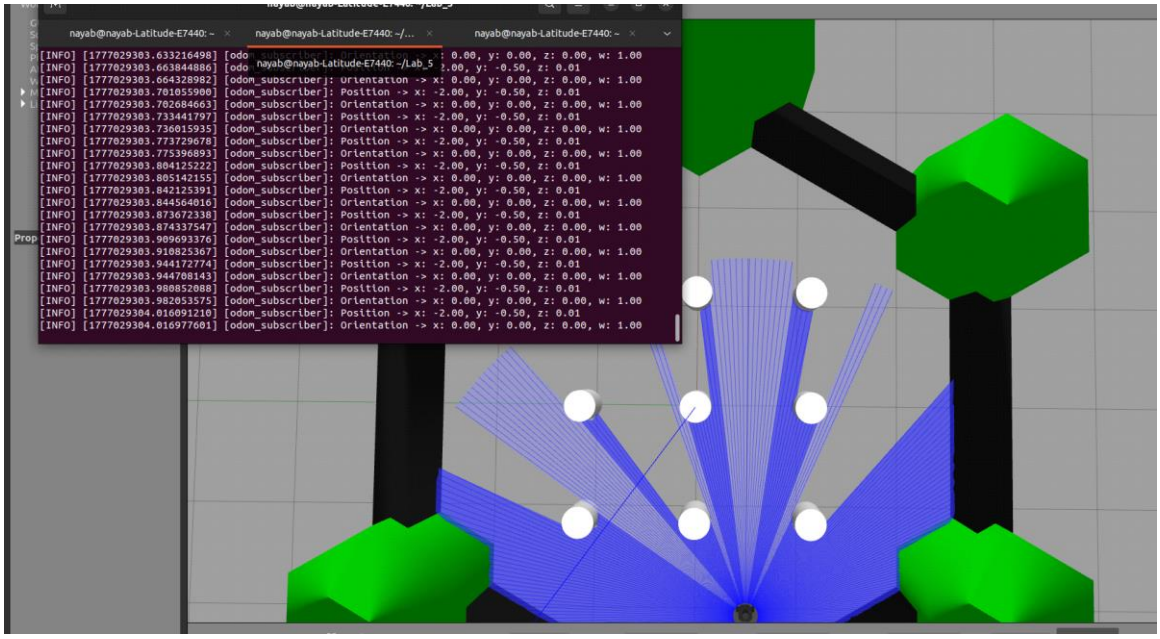
Code Breakdown:

- Message Type: nav_msgs/msg/Odometry — imported from nav_msgs.msg.
- Subscriber: create_subscription() registers the callback to be called every time a new /odom message is received.
- msg.pose.pose.position: Accesses the nested position data inside the Odometry message (note the double .pose — the outer is PoseWithCovariance and the inner is Pose).
- msg.pose.pose.orientation: Accesses the quaternion orientation with components x, y, z, w.
- :.2f format: Rounds the float values to 2 decimal places for clean output.
- rclpy.spin(node): Keeps the node running and listening for incoming messages indefinitely.

Running Output:

```
[INFO] [odom_subscriber]: Odom Subscriber Node Started
[INFO] [odom_subscriber]: Position -> x: 0.00, y: 0.00, z: 0.00
[INFO] [odom_subscriber]: Orientation -> x: 0.00, y: 0.00, z: 0.00, w: 1.00
[INFO] [odom_subscriber]: Position -> x: 0.04, y: 0.00, z: 0.00
[INFO] [odom_subscriber]: Orientation -> x: 0.00, y: 0.00, z: 0.01, w: 1.00
[INFO] [odom_subscriber]: Position -> x: 0.11, y: 0.00, z: 0.00
[INFO] [odom_subscriber]: Orientation -> x: 0.00, y: 0.00, z: 0.00, w: 1.00
[INFO] [odom_subscriber]: Position -> x: 0.19, y: 0.01, z: 0.00
[INFO] [odom_subscriber]: Orientation -> x: 0.00, y: 0.00, z: 0.02, w: 1.00
...
```





6. Observations on Expected vs. Simulated Motion

A key observation during this lab was the comparison between expected robot motion (as commanded) and the actual simulated motion reported by odometry. Several points were noted:

- When the robot was commanded to move in a straight line (linear.x = 0.2, angular.z = 0.0), the Gazebo simulation produced a nearly perfect straight path. However, in a real physical robot, wheel slip, floor friction, and motor noise would cause the robot to drift from the straight line.
- The odometry values at startup were (0, 0, 0) as expected, confirming that the /odom frame is reset to the robot's initial spawn position.
- When attempting to return to (0, 0, 0) in Task 7, small discrepancies in the odometry readings were observed. This is expected even in simulation due to floating-point precision in the physics engine and minor integration errors in the velocity-to-position computation.
- The orientation quaternion at startup had w = 1.0 and x, y, z = 0.0, which corresponds to zero rotation — the robot was facing the positive X direction. As the robot turned, the oz and ow values changed, reflecting rotation around the Z-axis (yaw).
- The LiDAR scan distance readings in RViz were consistent with the visual distances in Gazebo, confirming that the simulated sensor model matches the physical geometry of the environment.

7. Conclusion

This lab introduced important simulation and visualization tools in the ROS 2 ecosystem for mobile robotics. Gazebo was used as a physics-based simulation environment for the TurtleBot3, while RViz was used to visualize sensor data such as LaserScan, TF, Odometry, Path, and Map in real time.

The Cartographer SLAM package enabled real-time mapping and localization of the robot within the simulated environment. Keyboard teleoperation was used to manually control the

robot and perform navigation tasks. Additionally, the ros2 bag tool was used to record and replay topic data for analysis and debugging.

Two custom ROS 2 Python nodes were developed: a publisher node that alternates robot velocity using a timer callback, and a subscriber node that reads and displays odometry data.

The main challenge was understanding the nested structure of the Odometry message and quaternion-based orientation representation. Once understood, the data interpretation became clear and accurate.

Overall, the lab provided a solid foundation in ROS 2 simulation, visualization, SLAM, and node development, which are essential for advanced mobile robotics applications.